

P3985R0

Concepts for `std::simd`

Published Proposal, 2026-01-28

This version:

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3985r0.html>

Author:

[Daniel Towner](#) (Intel Corporation)

Audience:

LEWG, LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

github.com/cplusplus/papers/issues

Abstract

This paper proposes adding 10 public concepts to the `std::simd` namespace, enabling clear template constraints for SIMD programming. These concepts provide a natural extension of the standard's concept vocabulary to SIMD types, with some concepts mirroring scalar type concepts like `std::integral` and `std::floating_point`, while others are specific to SIMD programming needs.

Table of Contents

1 Introduction and Motivation

1.1 The Problem

1.2 The Solution

1.3 Current State

1.4 Proposal Overview

2 Proposed Concepts

2.1 Design Principle

2.2 Concept Definitions

3 Design Decisions

3.1 Namespace: Why `std::simd`?

3.2 Naming: `vec_*` vs `simd_*`

3.3 Relationship to Exposition-Only Concepts

4 Implementation Experience

5 Relationship to SIMD-Generic Programming

6 Proposed Wording

6.1 Wording Strategy

6.2 Wording Sketch

References

Informative References

§ 1. Introduction and Motivation

§ 1.1. The Problem

The C++26 standard library includes `std::simd::basic_vec<T, Abi>` for data-parallel programming ([[simd](#)]). Yet it lacks public concepts for constraining SIMD types in templates, forcing verbose and repetitive constraints:

EXAMPLE 1

```
template<typename T, typename ABI>
    requires std::integral<T>
constexpr std::simd::basic_vec<T, ABI>
add_sat(const std::simd::basic_vec<T, ABI>& lhs,
        const std::simd::basic_vec<T, ABI>& rhs) noexcept;
```

The element type constraint sits separated from the vector type, and the template parameter list gets cluttered with both element and ABI parameters.

§ 1.2. The Solution

We propose 10 public concepts for `std::simd`, such as `vec_integral`, `vec_complex`, `vec_of`, and more. The concepts focus on element types rather than vector sizes or relationships between types. SIMD algorithms depend on element type (e.g., which operators or function calls are valid), but rarely on size, which varies by platform. This element-centric approach serves portability well since developers care about how operations work, not how many elements get processed.

For masks, type detection as a mask alone seems to suffice. We've never found a need to provide concepts related to mask which do any more than that. If necessary we can deduce the mask type from any related concepts in the same function. For example:

```
template<vec_complex V>
constexpr auto do_complex_op(V p0,
                             typename V::mask_type p1)
{
    // p1 is a mask type related to V and doesn't need its own concept.
}
```

§ 1.3. Current State

The C++26 working draft ([[simd](#)]) uses six exposition-only concepts in the specification: *simd-type*, *simd-mask-type*, *simd-floating-point*, *simd-signed-integral*, *simd-unsigned-integral*, and *simd-complex*. These serve the specification but aren't available to users. Our proposal makes similar concepts public and extends the vocabulary beyond what the spec currently needs.

§ 1.4. Proposal Overview

We propose 10 public concepts organized into three categories:

1. Type Detection (3 concepts): Identify SIMD types

- `vec_type<V>` - any `std::simd::basic_vec<T, Abi>`
- `mask_type<M>` - any `std::simd::basic_mask<Bytes, Abi>`

- `vec_or_mask_type<V>` - either vec or mask

2. Element Type Refinements (6 concepts): Constrain by element category

- `vec_integral<V>` - vector with integral elements
- `vec_signed_integral<V>` - vector with signed integral elements
- `vec_unsigned_integral<V>` - vector with unsigned integral elements
- `vec_floating_point<V>` - vector with floating-point elements
- `vec_complex<V>` - vector with complex number elements
- `vec_arithmetic<V>` - vector with integral OR floating-point (excludes complex)

3. Element Type Matching (1 concept): Exact element type

- `vec_of<V, T>` - vector type `V` with specific element type `T`

§ 2. Proposed Concepts

§ 2.1. Design Principle

Several of these concepts extend the scalar type concept pattern to SIMD. Just as `std::integral<T>` describes integral types, `std::simd::vec_integral<V>` describes vec types with integral elements. The same pattern applies to `signed_integral`, `unsigned_integral`, and `floating_point`. Other concepts like `vec_complex`, `vec_arithmetic`, and `vec_of` address needs specific to SIMD programming.

§ 2.2. Concept Definitions

The proposed concepts are straightforward compositions of type detection and standard library concepts:

Concept	Definition	Notes
<code>vec_type<V></code>	<code>V</code> is <code>basic_vec<T, Abi></code>	Detects any SIMD vector type
<code>mask_type<M></code>	<code>M</code> is <code>basic_mask<Bytes, Abi></code>	Detects any SIMD mask

		type
<code>vec_or_mask_type<V></code>	<code>vec_type<V> mask_type<V></code>	Either vector or mask
<code>vec_integral<V></code>	<code>vec_type<V> && integral<typename V::value_type></code>	Vector with integral elements
<code>vec_signed_integral<V></code>	<code>vec_type<V> && signed_integral<typename V::value_type></code>	Vector with signed integral elements
<code>vec_unsigned_integral<V></code>	<code>vec_type<V> && unsigned_integral<typename V::value_type></code>	Vector with unsigned integral elements
<code>vec_floating_point<V></code>	<code>vec_type<V> && floating_point<typename V::value_type></code>	Vector with floating-point elements
<code>vec_complex<V></code>	<code>vec_type<V> && is-complex<typename V::value_type></code>	Vector with complex elements
<code>vec_arithmetic<V></code>	<code>vec_type<V> && (integral<typename V::value_type> floating_point<typename V::value_type>)</code>	Excludes complex; for operations like <code>copysign</code>
<code>vec_of<V, T></code>	<code>vec_type<V> && same_as<typename V::value_type, T></code>	Exact element type; <code>V</code> first for constrained-auto

§ 3. Design Decisions

§ 3.1. Namespace: Why `std::simd`?

These concepts belong in the `std::simd` namespace following established standard library practice. Range concepts live in `std::ranges` (like `std::ranges::range` and `std::ranges::input_range`), and SIMD types already live in `std::simd` (`std::simd::vec` and `std::simd::mask`). Placing SIMD concepts in the same namespace maintains this organizational pattern.

§ 3.2. Naming: `vec_*` vs `simd_*`

The `vec_` prefix follows from the type name itself. Since the primary type alias is `std::simd::vec<T, Abi>`, a concept like `vec_integral` naturally reads as "a vec with integral elements." This parallels how scalar concepts work: `std::integral<T>` describes an integral type, so `std::simd::vec_integral<V>` describes a vec type with integral elements.

Matthias Kretz's [P3287R2] explored an alternative naming approach using unprefix names like `simd::integral`, `simd::floating_point`, etc. This approach is elegant and concise. However, it raises several questions for vec-specific concepts. Would `simd::complex` clash semantically with the type `std::complex`? Does `simd::arithmetic` mean "SIMD types with arithmetic elements" or "arithmetic types that may include SIMD"? What would `vec_of<T>` be called (perhaps `simd::exact_element_type<T>`)?

We believe the `vec_` prefix provides clarity and avoids ambiguity, particularly for concepts that don't have direct scalar equivalents. The slightly increased verbosity seems justified by the reduced potential for confusion. The relationship to future `simd_generic` work is discussed in [§5 Relationship to SIMD-Generic Programming](#).

§ 3.3. Relationship to Exposition-Only Concepts

The C++26 draft includes six exposition-only concepts:

- *simd-type*
- *simd-mask-type*
- *simd-floating-point*
- *simd-signed-integral*

- *simd-unsigned-integral*
- *simd-complex*

The public concepts proposed here parallel these exposition-only concepts. The formal wording can either replace the exposition-only concepts with public equivalents, or define the public concepts separately while retaining the exposition-only versions for specification use. We leave this decision to LWG based on what best serves the specification.

§ 4. Implementation Experience

These concepts have been implemented and used in Intel’s SIMD reference implementation, which is deployed in production DSP workloads. The implementation confirms that SIMD algorithms naturally focus on element operations rather than vector size. Element type determines which operations are valid and which functions can be called, while size is typically a platform detail that doesn’t affect algorithm logic. This observation holds across diverse use cases from saturating arithmetic to complex number operations.

Hardware intrinsics have proven particularly dependent on exact element types. AVX-512 FP16 operations, for instance, require precisely `_Float16` rather than any other floating-point type, even though vector size may vary. The `vec_of` concept addresses this need cleanly.

The implementation burden is minimal. Each concept composes existing type detection with standard library scalar concepts, requiring no new compiler support and imposing zero runtime cost. The design follows established patterns from `<concepts>`, making it immediately familiar to developers already using concepts in their code.

§ 5. Relationship to SIMD-Generic Programming

Matthias Kretz’s [P3287R2] explored the concept of a `std::simd_generic` namespace containing unified scalar/SIMD concepts. This would enable writing fully generic algorithms that work transparently with both scalar and SIMD types, such as a `simd_generic::integral` concept satisfied by both `int` and `vec<int>`. Such a facility would allow developers to write code that automatically benefits from SIMD when available while falling back to scalar operations otherwise. That paper also explored unprefix naming within `std::simd`, such as `simd::integral` and `simd::floating_point`.

This paper deliberately proposes only the concepts specific to `std::simd`, not `simd_generic`. SIMD-specific concepts and unified scalar/SIMD concepts serve different purposes and have different

scopes. The `simd_generic` facility requires broader design decisions affecting more of the standard library, while these SIMD concepts can be added now and provide immediate value. Developers who need unified concepts today can write them using the concepts proposed here as building blocks.

The `vec_` prefix in our naming supports clear layering with future `simd_generic` work. If `simd_generic::integral` eventually describes "scalar or SIMD integral types", then `simd::vec_integral` unambiguously refers to the vec-specific concept. The prefix signals that these concepts specifically constrain `basic_vec` instantiations, not a broader category.

The concepts proposed here are compatible with a future `simd_generic` namespace. A `std::simd_generic::integral<T>` concept would naturally compose with `std::simd::vec_integral<V>`, with no conflicts or overlaps. The unified concepts would simply be defined in terms of these SIMD-specific concepts, creating a natural layering. We view `simd_generic` as important future work, but not a prerequisite for standardising SIMD-specific concepts.

§ 6. Proposed Wording

§ 6.1. Wording Strategy

We present two possible approaches for the formal wording, leaving the choice to LWG:

Approach A: Replace Exposition-Only Concepts

- Remove *simd-type*, *simd-floating-point*, *simd-complex* (and others) from [simd.syn]
- Add public `vec_type`, `vec_floating_point`, `vec_complex` (and others) to [simd.syn]
- Update all specification uses of exposition-only concepts to use public ones

Approach B: Keep Both

- Retain exposition-only concepts for specification use
- Add public concepts as additional vocabulary
- No changes to existing specification text

Recommendation: We slightly prefer Approach A to avoid the redundancy in having public and exposition concepts which say the same thing, but acknowledge that it may require more extensive changes to the specification text.

§ 6.2. Wording Sketch

The following is a sketch of the proposed wording. Complete wording will be developed based on LWG feedback on the approach.

Add to **[simd.syn]**:

EXAMPLE 2

```
namespace std::simd {
    // [simd.concepts], concepts
    template<class V>
    concept vec_type = /* see below */;

    template<class M>
    concept mask_type = /* see below */;

    template<class V>
    concept vec_or_mask_type = vec_type<V> || mask_type<V>;

    template<class V>
    concept vec_integral = vec_type<V> && integral<typename V::value_type>;

    template<class V>
    concept vec_signed_integral = vec_type<V> && signed_integral<typename V::value_type>;

    template<class V>
    concept vec_unsigned_integral = vec_type<V> && unsigned_integral<typename V::value_type>;

    template<class V>
    concept vec_floating_point = vec_type<V> && floating_point<typename V::value_type>;

    template<class V>
    concept vec_complex = vec_type<V> && /* V::value_type is complex<T> */;

    template<class V>
    concept vec_arithmetic = vec_type<V> &&
        (integral<typename V::value_type> || floating_point<typename V::value_type>);

    template<class V, class T>
    concept vec_of = vec_type<V> && same_as<typename V::value_type, T>;
}
```

[Additional detailed specification for each concept would follow the pattern above.]

§ References

§ Informative References

[P3287R2]

Matthias Kretz. *Exploration of namespaces for `std::simd`*. 2024. URL: <https://wg21.link/P3287R2>

[SIMD]

Data-parallel types. URL: <https://eel.is/c++draft/simd>